Model API & Containerization Guide

FastAPI Service Packaging, Dockerization & Testing Verification

This document details the complete end-to-end implementation for wrapping a trained Machine Learning model into a production-ready HTTP API using FastAPI, containerizing it with Docker, and running inference tests locally.

1. Repository Structure

Organize the project directory for the repository as follows:

```
ml-model-api/
├── app/
│   ├── __init__.py
│   ├── main.py          # FastAPI application & endpoints
│   └── model.py         # Model loading and inference wrapper
├── Dockerfile           # Container build instructions
├── requirements.txt     # Python dependencies
└── README.md            # Execution and deployment documentation
```

2. Implementation Files

requirements.txt

```
fastapi==0.110.0
uvicorn==0.28.0
pydantic==2.6.4
numpy==1.26.4
scikit-learn==1.4.1.post1
```

app/main.py

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from typing import List
import numpy as np

app = FastAPI(
    title="ML Model Inference API",
    description="REST API for serving machine learning predictions.",
    version="1.0.0"
)

class InferenceRequest(BaseModel):
    features: List[float] = Field(..., example=[5.1, 3.5, 1.4, 0.2])

class InferenceResponse(BaseModel):
    prediction: int
    probabilities: List[float]
    status: str

@app.get("/health")
def health_check():
    return {"status": "healthy", "service": "ml-inference-api"}

@app.post("/predict", response_model=InferenceResponse)
def predict(payload: InferenceRequest):
    try:
        data = np.array(payload.features).reshape(1, -1)
        # Mock model logic (replace with loaded model: model.predict(data))
        prediction = int(np.argmax(data) % 3)
        probabilities = [0.70, 0.20, 0.10]
        return InferenceResponse(
            prediction=prediction,
            probabilities=probabilities,
            status="success"
        )
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

3. Docker Configuration

Dockerfile

```
FROM python:3.10-slim

WORKDIR /app

# Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY ./app ./app

EXPOSE 8000

# Run uvicorn server
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```